for observability, turning what the system does into numbers, which you can then sum up and compare as time goes on. They are lightweight, can scale well, and are great when you want to watch systemwide trends in a clean way.

Prometheus is now basically the default choice for metrics collection in cloud native setups. The whole pull-based idea means it scrapes metrics from services that change a lot, which is why it works so well for Kubernetes clusters, and all that dynamic stuff around them.

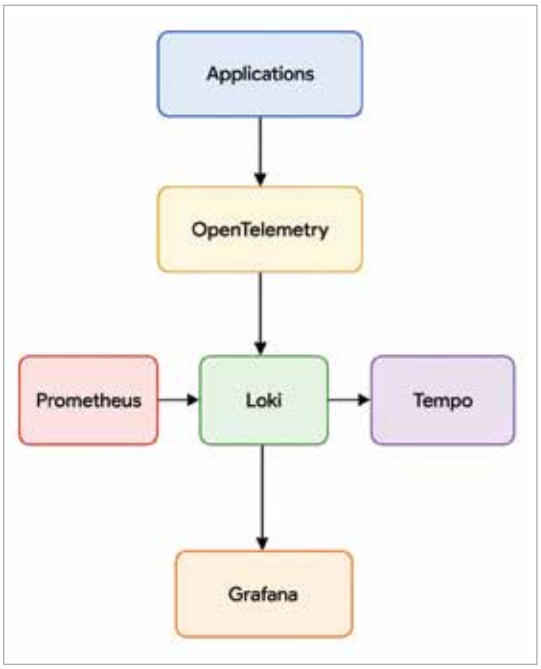

Figure 1: A common observability setup

```
http_requests_total 15432
request_duration_seconds
0.182
active_sessions 2341
```

But it doesn't stop at infrastructure monitoring. Modern telemetry stacks also pull in business-oriented metrics, like transaction volumes, user interaction signals, and reliability related indicators. That gives engineering teams a way to line up technical performance with actual business results, not just guesswork.

For instance, an online store might examine how added delay affects checkout completion. Or a media service could compare buffering frequency with user retention and see the relationship instead of relying on intuition.

Still, as systems become more fluid and spread across more nodes, metrics by themselves get a bit thin. You typically need logs too, so you can get the context behind those numerical patterns and understand what's happening.

From log chaos to actionable insights

Any time a service talks with another service—API requests, database lookups, authentication tries—it usually leaves log trails behind. In today's distributed setups this piles up fast, like terabytes each day, so just staring at raw logs is basically not workable.

The real pain isn't gathering the logs, it's making sense of them. Plain text logs tend to be kind of messy, so during an incident engineers end up scrolling, searching, and re-scanning unstructured stuff, which is hard enough on a normal day, but worse when everyone is under pressure. That reactive workflow drags out diagnosis, and can add up to longer downtime.

Observability today tries to push logs towards structured, queryable events. Once log formats are standardised, the whole system can correlate what's happening across different components, over time, instead of treating everything like separate islands.

```
{
  "timestamp": "2026-06-06T10:15:00Z",
  "service": "payment-api",
  "level": "ERROR",
  "event": "transaction_failed",
  "user_id": "7821",
  "trace_id": "abc-123",
}
```

And when those logs are enriched with trace identifiers, they suddenly feel way more useful. You can jump from a metric oddity straight to a trace, and then further down into the exact log entries that explain what went sideways.

In real systems, like fintech or healthcare apps, structured logging shortens the mean time to resolution because it gives sharp contextual hints. Instead of reading logs by hand, teams can just slice the data by service, request, or transaction ID and move on.

Finally, as log records get more structured and more connected, they naturally feed distributed tracing tools. Those tools then stitch together individual events into a full 'request journey', so the story of the failure becomes easier to follow, even when it spans many services.

Following the digital breadcrumbs: Distributed tracing with OpenTelemetry

In distributed architectures, one user request usually wanders across multiple services before it's done. But if you do not get visibility into that whole trip, figuring out why latency spikes or where a failure happens gets hard.

Distributed tracing helps by following a request as it spreads through the different services. Each step gets logged as a span, and those spans together turn into a trace that shows the full transaction story from start to finish.

OpenTelemetry has become the *de facto* standard for creating and exporting trace data. It gives you one common framework that works across languages, platforms, and even across different cloud setups.

```
from opentelemetry import trace
tracer = trace.get_tracer(__name__)
with tracer.start_as_current_
```